Library Database Management System

Relational Schema Document — Project Part 4

Team Administrator: Parker | Date: October 22, 2026

1. Introduction

1.1 Project Overview

The purpose of this project is to design and implement a relational database system to support a library. The database stores information about the collection, its members, and its staff. It supports catalog searches, account management, checkouts, check-ins, reservations, and circulation reports through separate member and administrative web interfaces. This document maps the ER model into nine relations for the PostgreSQL database hosted on Neon.

1.2 Scope

The core project includes a populated catalog, member and administrative interfaces, and queries for searching, inserting, updating, and deleting records. This schema defines account data, shared catalog attributes, media subtypes, checkout history, reservations, and librarian management assignments. Each checkout and reservation concerns one member and one media item. Reservation and reporting data are included to support the expanded functions described in the requirements document.

This schema assumes one lendable record per book ISBN. Multiple-copy inventory, subject classification, and notification delivery require further design work. Fines, payments, supplier records, branch management, and digital licensing are outside this version. The web interfaces remain part of the overall project; this report defines their supporting data rather than their screen layouts.

1.3 Glossary

Term	Definition
Relation and tuple	A table and one row within that table.
Domain	The permitted values and representation of an attribute.
Primary key or PK	The selected minimal set of attributes that uniquely identifies a row.
Foreign key or FK	An attribute set whose values must match a referenced key.
Candidate key	A minimal attribute set that can uniquely identify a row.
Composite key	A key consisting of more than one attribute.
Functional dependency or FD	X → Y means equal X values require equal Y values.
Referential integrity	Every non-null foreign key identifies an existing referenced row.
Subtype	A specialized category of a general entity, such as Book within Media_Item.
3NF and BCNF	Normal forms restricting dependencies to reduce redundancy; BCNF requires every nontrivial FD determinant to be a superkey.
Media Item	Any physical item in the library's circulating collection that can be checked out.

Term	Definition
Member	A registered library patron who can search the catalog and check out, check in, or reserve items.
Admin	A staff user with elevated privileges to add, edit, or remove records and generate reports.
Checkout	The process of a member borrowing a media item, recorded with a due date.
Check-in	The process of returning a previously checked-out item to the library.
Reservation	A request placed by a member to reserve an item that is currently checked out.
UI	User Interface — the web-based front end through which users interact with the database.
CRUD	Create, Read, Update, Delete — the four basic database operations.
ORM	Object-Relational Mapping — a technique/library used to interact with the database from application code.

2. Relational Schema Mapping

The mapping uses the standard entity, binary relationship, and specialization rules. The following decisions explicitly account for all entities and relationship connections in the conceptual model.

ER construct	Relational mapping
Patron and Librarian	Create PATRON and LIBRARIAN. Retain patron_id and admin_id as their primary keys.
Media_Item	Create MEDIA_ITEM with media_id as its primary key and the shared catalog attributes. Move isbn to BOOK.
Book subtype	Create BOOK with isbn as its primary key, following the instructor feedback. media_id is a required, unique foreign key to MEDIA_ITEM, so it is also a candidate key.
Magazine subtype	Create MAGAZINE with media_id as both its full primary key and foreign key. Remove the separate Item_id; no weak entity or partial key is used.
Movie subtype	Create MOVIE with media_id as both its primary key and foreign key. Its membership row records the subtype even without additional attributes.
Checkout and Reservation	Create separate transaction relations with their original identifiers as primary keys. Rename item_id to media_id for consistent references.
Four one to many relationships	Represent Patron–Checkout, Patron–Reservation, Media_Item–Checkout, and Media_Item–Reservation with required foreign keys in the transaction relations.
Librarian manages Media_Item	Map the many to many relationship to MANAGES. Its primary key is (admin_id, media_id); both attributes are foreign keys.

2.1 Relationship participation

Ranges describe the number of related rows for one instance of the named entity. Optional participation is assumed unless stated otherwise. These min..max ranges replace the original single-number and M/N labels.

Named relationship	Participation rules
Patron makes Reservation	Patron: 0..N reservations. Reservation: 1..1 patron.
Patron has Checkout	Patron: 0..N checkouts. Checkout: 1..1 patron.
Reservation reserves Media_Item	Media_Item: 0..N reservations. Reservation: 1..1 item.
Checkout references Media_Item	Media_Item: 0..N historical checkouts. Checkout: 1..1 item.
Librarian manages Media_Item	Librarian: 0..N items. Media_Item: 0..N librarians.
Media_Item specialization	Each subtype row: 1..1 parent. Parent: 0..1 row in each subtype, and exactly one subtype overall.

3. Relation Definitions

PK marks a primary key; FK marks a foreign key; UQ marks a unique constraint. Every primary key is NOT NULL. All foreign keys in this design are also NOT NULL. The dictionary specifies the remaining nullability and domains.

3.1 PATRON

patron_id [PK], first_name, last_name, email [UQ], password_hash, account_status, membership_date, phone

3.2 LIBRARIAN

admin_id [PK], first_name, last_name, email [UQ], password_hash, role, hire_date

3.3 MEDIA_ITEM

media_id [PK], title, author, publisher, publication_date, media_type, status, date_added

3.4 BOOK

isbn [PK], media_id [FK, UQ]

media_id → MEDIA_ITEM.media_id

3.5 MAGAZINE

media_id [PK, FK], issue_number, volume, frequency

media_id → MEDIA_ITEM.media_id

3.6 MOVIE

media_id [PK, FK]

media_id → MEDIA_ITEM.media_id

3.7 CHECKOUT

checkout_id [PK], patron_id [FK], media_id [FK], checkout_date, due_date, return_date

patron_id → PATRON.patron_id; media_id → MEDIA_ITEM.media_id

3.8 RESERVATION

reservation_id [PK], patron_id [FK], media_id [FK], reservation_date, status

patron_id → PATRON.patron_id; media_id → MEDIA_ITEM.media_id

3.9 MANAGES

admin_id [PK, FK], media_id [PK, FK]

admin_id → LIBRARIAN.admin_id; media_id → MEDIA_ITEM.media_id

MANAGES has one composite primary key, not two independent primary keys. BOOK uses ISBN as its primary key and retains a unique parent reference so that one media item cannot identify multiple book rows. Patron and librarian email addresses are unique within their respective tables, not across both tables.

4. Functional Dependencies and Normalization

These dependencies describe the intended real-world constraints, not coincidences in sample data. An arrow with several attributes on its right represents one FD for each attribute. Trivial dependencies are omitted.

Relation	Functional dependencies
PATRON	patron_id → first_name, last_name, email, password_hash, account_status, membership_date, phone; email → patron_id. Both patron_id and email are candidate keys.
LIBRARIAN	admin_id → first_name, last_name, email, password_hash, role, hire_date; email → admin_id. Both admin_id and email are candidate keys.
MEDIA_ITEM	media_id → title, author, publisher, publication_date, media_type, status, date_added
BOOK	isbn → media_id; media_id → isbn. Both are candidate keys.
MAGAZINE	media_id → issue_number, volume, frequency
MOVIE	No nontrivial FDs. media_id is the sole attribute and candidate key.
CHECKOUT	checkout_id → patron_id, media_id, checkout_date, due_date, return_date
RESERVATION	reservation_id → patron_id, media_id, reservation_date, status
MANAGES	No nontrivial FDs. The pair (admin_id, media_id) is the candidate key.

4.1 Normalization considerations

Under the listed dependencies, all nine relations satisfy BCNF and therefore 3NF: each determinant of a nontrivial FD is a candidate key. MANAGES has no non-key attributes, so it introduces no partial dependency. MOVIE is a single-attribute relation. BOOK has two candidate keys, and both of its nontrivial dependencies have a key on the left.

This conclusion depends on the stated business assumptions. Names, phone numbers, titles, publishers, and publication dates are not assumed to be unique. Magazine volume and issue number are not assumed globally unique. Frequency is stored for an individual magazine record without introducing a separately identified magazine series.

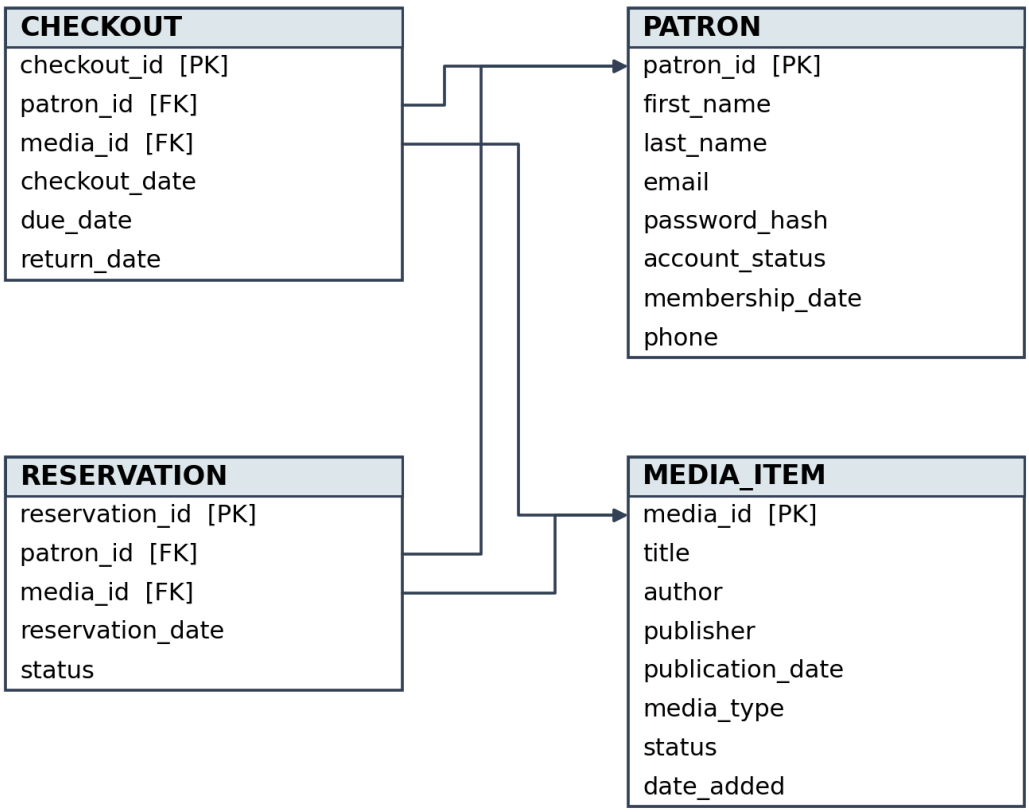
The author field stores one primary credited author or editor, not a comma-separated collection of authors. Supporting multiple independently searchable contributors would require a CONTRIBUTOR relation and a junction relation. An operational status derived from loans is not stored: MEDIA_ITEM.status records physical or administrative condition, while loan availability is calculated from active checkouts.

Future support for multiple physical copies requires an edition-and-copy design rather than weakening ISBN uniqueness. Normalization should be reassessed whenever new attributes or business dependencies are introduced.

5. Relational Schema Diagram

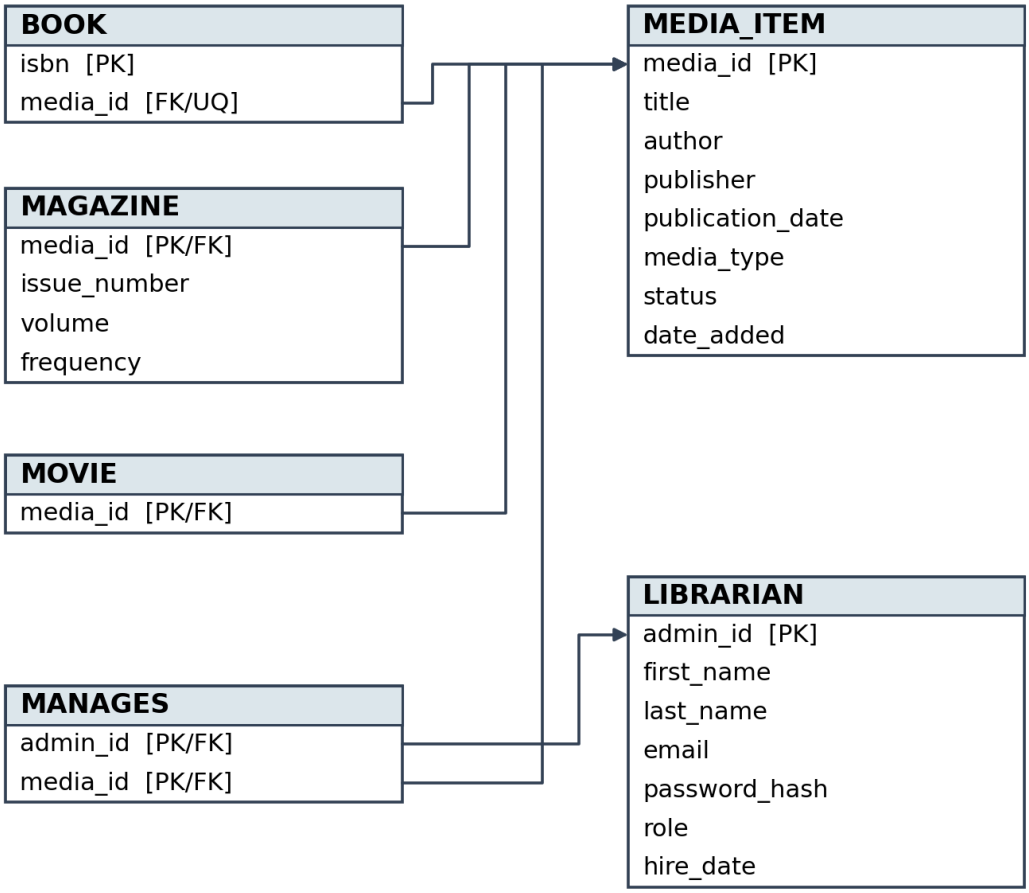
5.1 Circulation and patron relations

Arrowheads point from a foreign-key attribute to the referenced primary-key attribute. The two panels together form the complete schema. MEDIA_ITEM is repeated for readability; it remains one relation.



PK = primary key; FK = foreign key; UQ = unique.

5.2 Catalog subtypes and librarian management



BOOK.isbn is its primary key. BOOK.media_id is unique and required. MAGAZINE and MOVIE inherit their parent identifier as PK/FK. MANAGES uses the composite PK (admin_id, media_id). All nine foreign-key references appear across the two panels.

6. Data Dictionary

Data types follow PostgreSQL, the database engine selected for deployment on Neon. Lengths are in characters. NULL means unknown or not applicable; an empty string is not a substitute. Text is trimmed. Generated integer identifiers use GENERATED ALWAYS AS IDENTITY. The stated date limits are application domains, not PostgreSQL storage limits. TIMESTAMP values are stored as UTC without a time-zone suffix.

6.1 PATRON

Attribute	Type	Null	Domain and constraints
patron_id	INT	No	PK; 1..2147483647; generated patron identifier.
first_name	VARCHAR(80)	No	1..80 characters; nonblank given name.
last_name	VARCHAR(80)	No	1..80 characters; nonblank family name.
email	VARCHAR(254)	No	UQ; 3..254 characters; validated email, trimmed and lowercased before storage.
password_hash	VARCHAR(255)	No	1..255 characters; encoded salted password hash; never plaintext.
account_status	VARCHAR(12)	No	One of active, suspended, closed; default active.
membership_date	DATE	No	Valid date 1000-01-01..9999-12-31; not later than insertion date.
phone	VARCHAR(25)	Yes	NULL or 1..25 characters; optional contact number; not unique.

6.2 LIBRARIAN

Attribute	Type	Null	Domain and constraints
admin_id	INT	No	PK; 1..2147483647; generated librarian identifier.
first_name	VARCHAR(80)	No	1..80 characters; nonblank given name.
last_name	VARCHAR(80)	No	1..80 characters; nonblank family name.
email	VARCHAR(254)	No	UQ; 3..254 characters; validated email, trimmed and lowercased before storage.
password_hash	VARCHAR(255)	No	1..255 characters; encoded salted password hash.
role	VARCHAR(15)	No	One of librarian, administrator; default librarian.
hire_date	DATE	No	Valid date 1000-01-01..9999-12-31; not later than insertion date.

6.3 MEDIA_ITEM

Attribute	Type	Null	Domain and constraints
media_id	INT	No	PK; 1..2147483647; generated catalog item identifier.
title	VARCHAR(255)	No	1..255 characters; nonblank title; not unique.
author	VARCHAR(160)	Yes	NULL or 1..160 characters; one primary author/editor credit; NULL if not applicable or unknown.
publisher	VARCHAR(160)	Yes	NULL or 1..160 characters; publisher or distributor name.
publication_date	DATE	Yes	NULL or valid date 1000-01-01..9999-12-31; publication or release date.
media_type	VARCHAR(8)	No	Exactly book, magazine, or movie; agrees with subtype membership.
status	VARCHAR(12)	No	One of circulating, repair, lost, withdrawn; default circulating. Not loan status.
date_added	DATE	No	Valid date 1000-01-01..9999-12-31; not later than insertion date.

6.4 BOOK

Attribute	Type	Null	Domain and constraints
isbn	CHAR(13)	No	PK; exactly 13 digits, prefix 978 or 979, valid ISBN-13 checksum; no spaces or hyphens.
media_id	INT	No	UQ and FK → MEDIA_ITEM.media_id; positive parent identifier.

6.5 MAGAZINE

Attribute	Type	Null	Domain and constraints
media_id	INT	No	PK and FK → MEDIA_ITEM.media_id; positive parent identifier.
issue_number	INT	No	1..2147483647; numeric issue within a volume.
volume	INT	No	1..2147483647; numeric volume identifier.
frequency	VARCHAR(16)	No	One of daily, weekly, biweekly, monthly, bimonthly, quarterly, semiannual, annual, irregular.

6.6 MOVIE

Attribute	Type	Null	Domain and constraints
media_id	INT	No	PK and FK → MEDIA_ITEM.media_id; positive parent identifier.

6.7 MANAGES

Attribute	Type	Null	Domain and constraints
admin_id	INT	No	Composite PK component; FK → LIBRARIAN.admin_id.
media_id	INT	No	Composite PK component; FK → MEDIA_ITEM.media_id.

6.8 Domain conventions

The frequency vocabulary uses biweekly for every two weeks and bimonthly for every two months. Issue and volume identifiers are restricted to positive integers in this scope. Named or combined issues require a later domain revision.

Only independent entity and transaction identifiers are generated. Foreign keys, subtype keys, and MANAGES key components reuse existing identifiers. Every referenced identifier has the same integer domain as its parent key.

6.9 CHECKOUT

Attribute	Type	Null	Domain and constraints
checkout_id	INT	No	PK; 1..2147483647; generated loan identifier.
patron_id	INT	No	FK → PATRON.patron_id; positive borrower identifier.
media_id	INT	No	FK → MEDIA_ITEM.media_id; positive item identifier.
checkout_date	TIMESTAMP	No	Valid UTC timestamp 1000-01-01 00:00:00..9999-12-31 23:59:59; loan start.
due_date	TIMESTAMP	No	Same timestamp domain; must be later than checkout_date.
return_date	TIMESTAMP	Yes	NULL for an active loan; otherwise same timestamp domain and ≥ checkout_date.

6.10 RESERVATION

Attribute	Type	Null	Domain and constraints
reservation_id	INT	No	PK; 1..2147483647; generated reservation identifier.
patron_id	INT	No	FK → PATRON.patron_id; positive reserving patron identifier.
media_id	INT	No	FK → MEDIA_ITEM.media_id; positive requested item identifier.
reservation_date	TIMESTAMP	No	Valid UTC timestamp 1000-01-01 00:00:00..9999-12-31 23:59:59; request creation.
status	VARCHAR(12)	No	One of pending, fulfilled, cancelled, expired; default pending.

7. Integrity Rules and Design Rationale

7.1 Declarative constraints

Use PRIMARY KEY, UNIQUE, NOT NULL, FOREIGN KEY, and CHECK constraints for the keys and row-local rules in the dictionary. Enforce positive identifiers, nonblank required strings, enumerated values, due_date > checkout_date, and return_date IS NULL OR return_date >= checkout_date. All nine foreign keys reference primary keys. Use ON DELETE RESTRICT and ON UPDATE RESTRICT for every foreign key; identifiers are immutable. Retain historical records and close accounts or withdraw items instead of deleting referenced parents.

7.2 Constraints requiring coordinated operations

Rule	Required enforcement approach
One active loan per item	Use a unique partial index on CHECKOUT(media_id) WHERE return_date IS NULL. Process checkout and item eligibility checks in one transaction.
One pending reservation per patron and item	Use a unique partial index on RESERVATION(patron_id, media_id) WHERE status = 'pending'. Historical resolved requests remain allowed.
Eligible borrowing and reservations	At creation, require an active patron and a circulating media item. Lock relevant patron/item rows in a consistent order. Checkout also requires no active loan; reservations may be placed while an item is on loan.
Total and disjoint specialization	Create the parent and exactly one matching subtype atomically. Enforce media_type agreement and prohibit rows in other subtype tables through a controlled database procedure or equivalent transaction service. FKs alone cannot enforce this rule.
Valid domain inputs	Validate email syntax, ISBN checksum, and insertion-date rules in the controlled write path. Store timestamps consistently in UTC.
Return and fulfillment handling	Returning an item fills return_date. Creating a checkout from a pending reservation changes that request to fulfilled in the same transaction. Historical reservations need not reference a checkout in this scope.

7.3 Data type and indexing rationale

ISBN uses CHAR(13) because it is a fixed-length identifier, not a quantity. VARCHAR supports variable-length names and descriptions. DATE captures calendar-only facts; TIMESTAMP preserves ordering of circulation events. A nullable return timestamp distinguishes active loans. Phone numbers are text to preserve prefixes and formatting.

In addition to key indexes, use CHECKOUT(media_id, return_date), CHECKOUT(patron_id), RESERVATION(media_id, status, reservation_date), RESERVATION(patron_id), and MANAGES(media_id). The MANAGES primary key already supports searches starting with admin_id. Indexes improve lookup performance but do not replace the transactional rules above.